

CMPT 354
Database Systems I

Introduction

Martin Ester
Simon Fraser University
Fall 2023

The Information Society



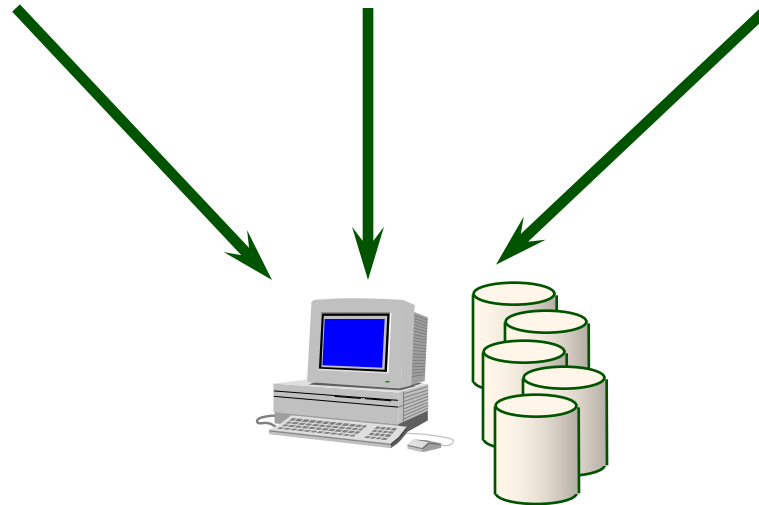
Human Genome



Customer Transactions



Online Bookstore



The Information Society



Facebook

- Captures more than 1.5PB weblog data daily.
- Recommends friend and items.
- Makes targeted ads.

Amazon

- Collects more than 200TB of weblog data daily.
- Recommends items.
- Makes targeted ads.

The Information Society



Large Hadron Collider

- For particle physics experiments, the largest experimental facility in the world.
- About 150 million sensors delivering data 40 million times per second.

Genomics

- Sequencing one human genome produces ~1TB of data.
- Precision Medicine Initiative: 1 million volunteers provide EHRs, healthcare claims, biological samples (e.g. for DNA collection).

Database Systems as a Solution



- Database management systems can manage very large collections of data, i.e. store, update, and retrieve the data.
- The market for Database management systems is huge, anticipated to reach US\$ 101 Billion per year by 2030.
- Leading commercial systems
 - Oracle
 - Microsoft SQL Server
 - IBM Db2

What is a database?



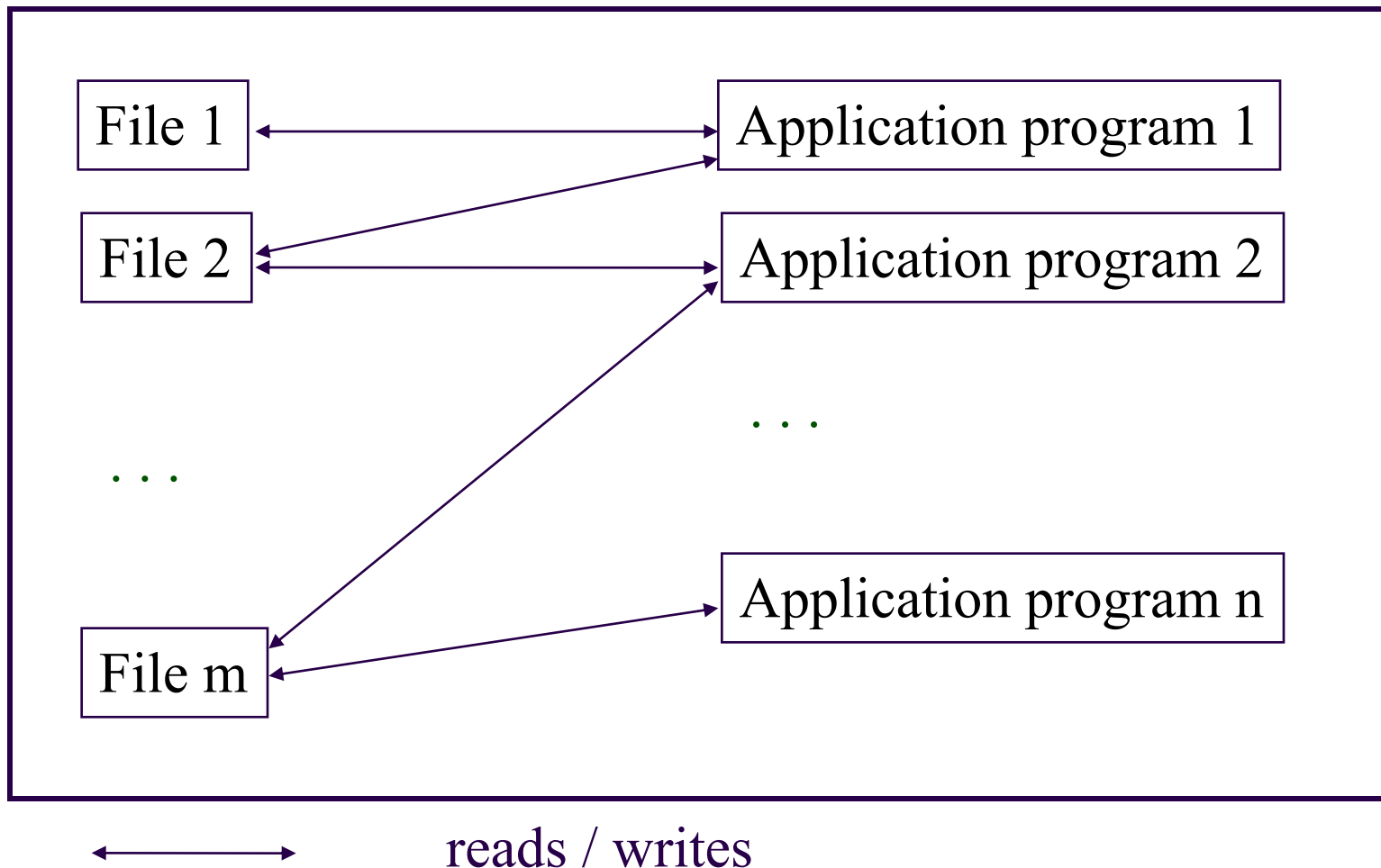
- A *database* (*DB*) is a very large, integrated, permanent collection of data.
- Models real-world *enterprise*.
 - Entities (e.g., students, courses)
 - Relationships (e.g., Madonna is taking CMPT354).
- Example databases:
 - Customer transactions
 - Human genomes
 - Online bookstore
 - ...

What is a DB(M)S?



- A *Database Management System (DBMS)* is a software package designed to store, manage and retrieve databases.
- A *Database System (DBS)* consists of two components:
 - the DBMS
 - the DB.
- A DBMS is a generic software that can manage databases for any application as long as they are in the proper format (data model).
- The DB is the application-specific content.

Data Storage Without DBMS

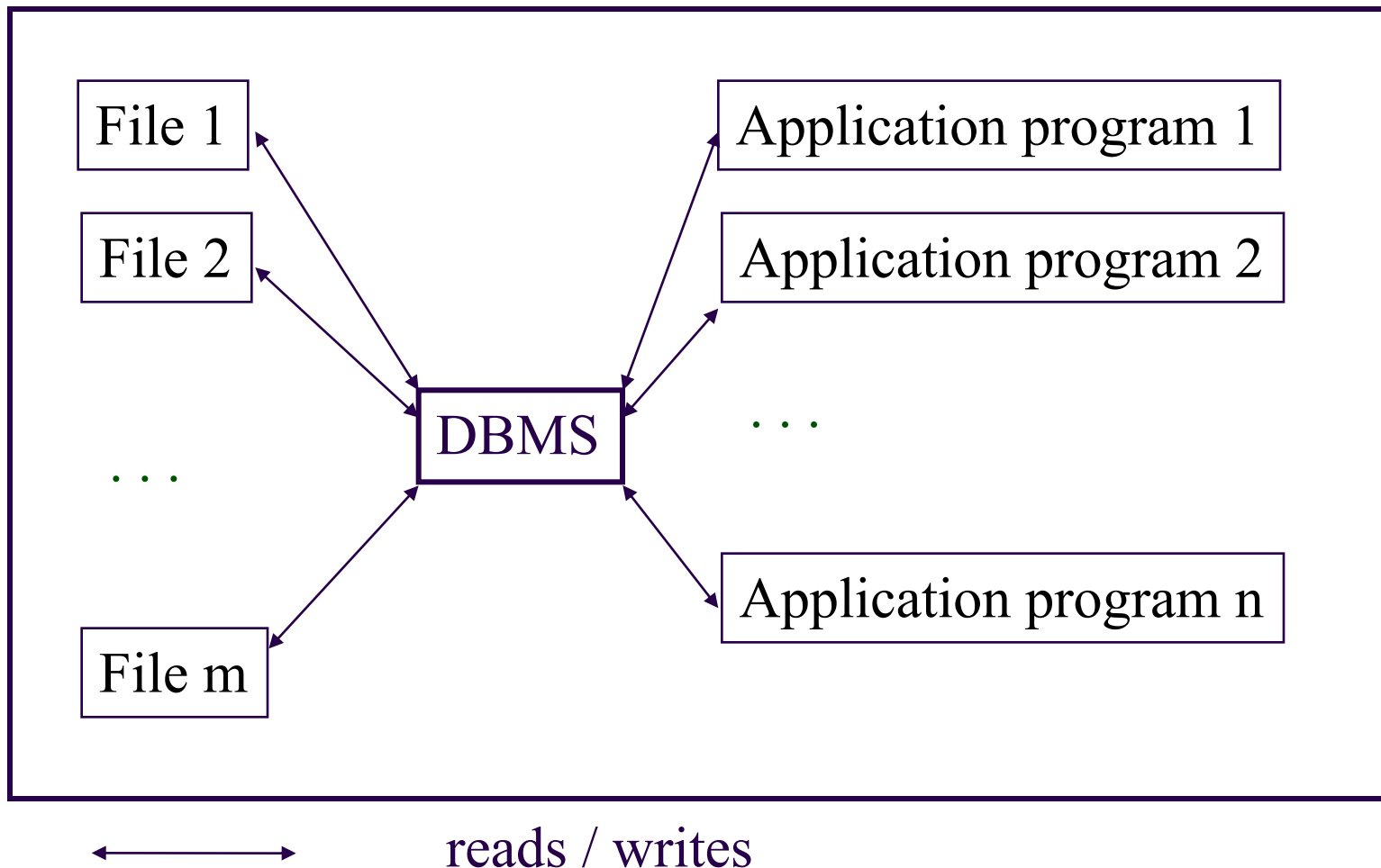


Data Storage Without DBMS



- Working directly with the file system creates major problems:
 - What about application program 2 if application program 1 adds one attribute to the records in file 2?
 - How to efficiently access only one out of one million records?
 - What if application programs 2 and n simultaneously want to access and modify file m?
 - How to restore a meaningful database state after a system crash during the run of an application program?

Data Storage With DBMS



Data Storage With DBMS



- All data access is centralized and managed by the DBMS.
- The DBMS provides:
 - Logical data independence.
 - Physical data independence.
 - Reduced application development time.
 - Efficient access.
 - Data integrity and security.
 - Concurrent access / concurrency control.
 - Recovery from crashes.

Data Models

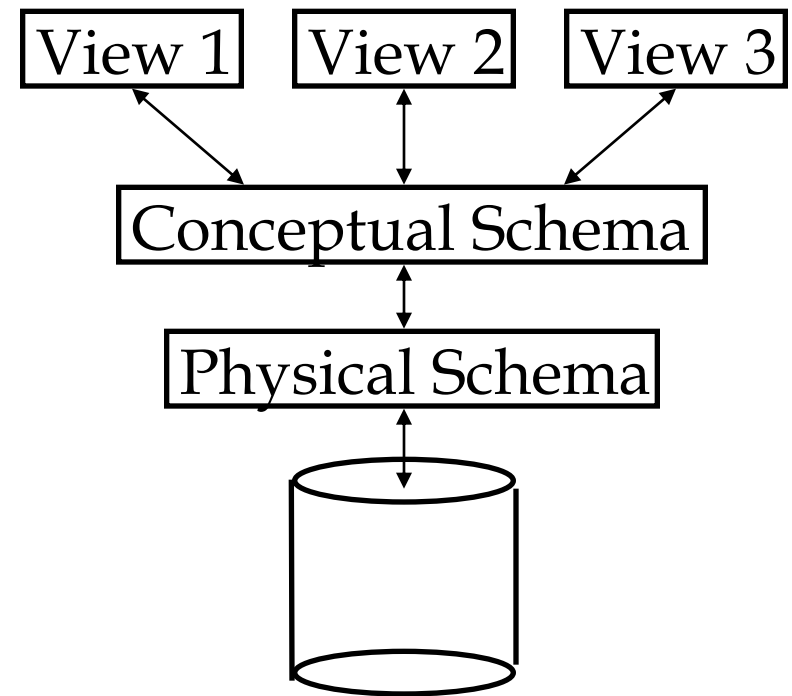


- A *data model* is a collection of concepts for describing data (a formal language!).
- A *schema* is a description of a particular collection of data (database), using the given data model.
- The *relational data model* is the most widely used model today.
 - Main concept: *relation*, basically a table with rows and columns.
 - Every relation has a schema, which describes the columns, or fields.

Levels of Abstraction



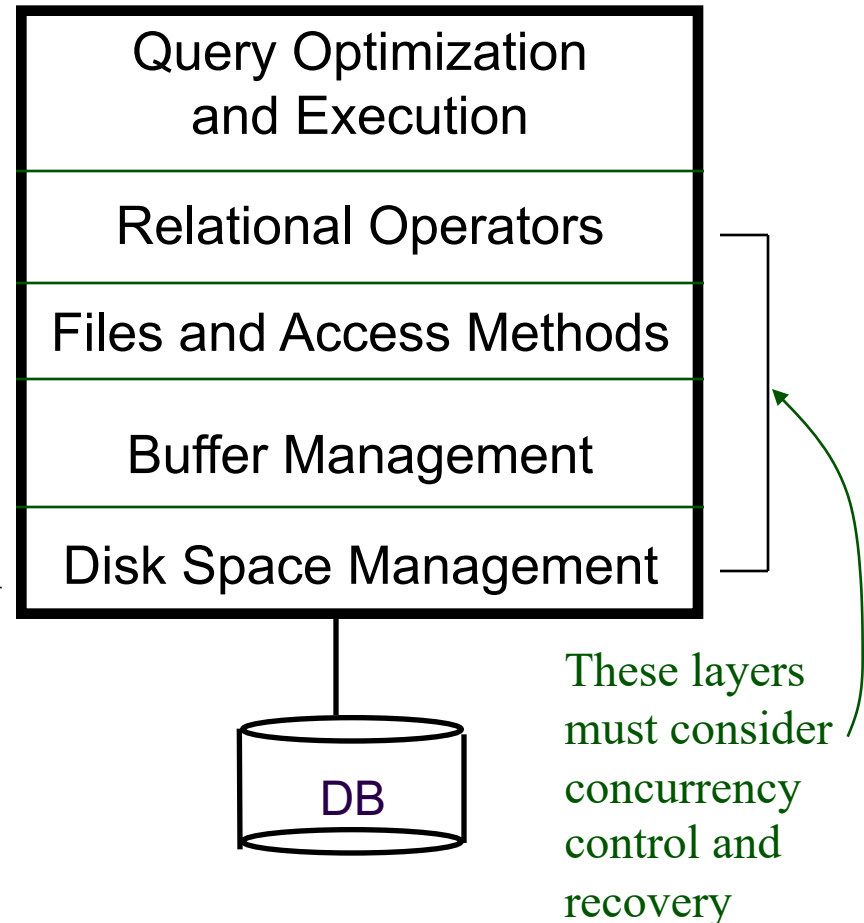
- The *conceptual schema* defines the logical structure of the whole database.
- An *external schema* (*view*) describes how some user sees the data (restricted access, derived data).
- The *physical schema* describes the storage and index structures of the database (the implementation).



Structure of a DBMS



- A typical DBMS has a layered architecture.
- The figure does not show the concurrency control and recovery components.
- This is one of several possible architectures; each system has its own variations.



Example: University Database



■ Conceptual schema:

*Students(sid: string, name: string, login: string,
age: integer, gpa: real)*

Courses(cid: string, cname: string, credits: integer)

Enrolled(sid: string, cid: string, grade: string)

■ Physical schema:

- Relations stored as unordered files.
- Index on first column of Students.

■ External schema (view):

- *Course_info(cid: string, enrollment: integer)*

Example: University Database



■ Updates:

- insert new student (*XXXid*, *XXX*, *XXX*, 21, 3.5)
- delete course *CMPT-YYY*
- enroll student *XXXid* in course *CMPT-ZZZ*

■ Queries:

- retrieve all students having a gpa of < 3.0
- retrieve the average gpa of all students enrolled in course *CMPT-ZZZ*
- retrieve the names of all courses having at least one student with a grade of 4.0

Data Independence



- The layered DBMS architecture insulates applications from how data is structured and stored.
- A DBS can be programmed at a much higher level of abstraction than the file system.
- Application programs need not be modified on change of database structure and / or storage.
- ➡ Reduced application development and maintenance time

Data Independence



- *Logical data independence*: Protection from changes in *logical* structure of data.
Ex.: adding another attribute to a relation
- *Physical data independence*: Protection from changes in *physical* structure of data.
Ex.: adding / removing index structure
or moving file to another disk

Efficient Access



- When the user wants to access only a small portion of a large relation, the DBS does not scan the entire relation.

Ex.: retrieve sid of all students enrolled in course CMPT-ZZZ

- An *index structure* maps (logical) attribute values to (physical) storage addresses.

Ex.: need index on attribute sid of relation Enrolled

- Index lookup returns the storage addresses of all matching tuples that can be directly accessed without scanning the whole relation.

➡ Much more efficient query processing

Concurrency Control



- *Concurrent execution of several user programs*
 - Many users want to work on the same database concurrently, cannot wait for other users to finish.
 - Because disk accesses are frequent, and relatively slow, it is important to keep the cpu humming by working on several user programs concurrently.
- Interleaving actions of different user programs can lead to inconsistency: e.g., check is cleared while account balance is being computed.
- DBMS ensures such problems don't arise: users can pretend they are using a single-user system.

Transactions



- A *transaction* is a sequence of database actions (reads/writes) with the following properties:
 - **Atomicity**: all-or-nothing property
 - **Consistency**: leaves the DB in a *consistent state* if DB is consistent when the transaction begins
 - **Isolation**: transaction is performed as if only one transaction at a time (serializable schedule)
 - **Durability**: effects of completed transactions are permanent
 - ➡ *ACID principle*

Ensuring Consistency



- Users can specify *integrity constraints* on the data, and the DBMS will enforce these constraints upon all database updates.
- Beyond this, the DBMS does not really *understand* the semantics of the data.
 - e.g., it does not understand how the interest on a bank account is computed.
- Thus, ensuring that a transaction (run alone) preserves consistency is ultimately the user's responsibility!

Ensuring Isolation



- DBMS ensures that concurrent (interleaved) execution of $\{T_1, \dots, T_n\}$ is equivalent to some *serial* execution of $\{T_1, \dots, T_n\}$.
- Before reading/writing an object, a transaction requests a *lock* on the object, and waits till the DBMS gives it the lock.
- Read locks are compatible with each other, but there can be only one write lock on an object at a given point of time.
- All locks are released at the end of the transaction.

Ensuring Isolation



- If an action of T_i (say, writing X) affects T_j (which perhaps reads X), one of them, say T_i , will obtain the lock on X first and T_j is forced to wait until T_i completes.
- This effectively orders the transactions.

Ensuring Atomicity / Durability



- DBMS ensures *atomicity* even if system crashes in the middle of a transaction.
- DBMS ensures *durability* also if system crashes after the commit of a transaction.
- Idea: Keep a *log* (history) of all relevant actions carried out by the DBMS while executing a set of transactions, i.e. log all updates and “transaction events” (commit, abort).

Crash Recovery



- A system crash may lead to the loss of information, that has not yet been flushed to the hard disk.
- A system crash can lead to partially executed transactions and inconsistent (disk-resident) databases.
- After a crash,
 - the effects of partially executed transactions are *undone* using the log, and
 - the effects of completely executed transactions are *redone* using the log.

Summary



- Datasets increasing in diversity and volume.
- DBMS used to manage and query large datasets.
- Benefits include recovery from system crashes, concurrent access, quick application development, data integrity and security.
- Levels of abstraction give data independence.
- A DBMS typically has a layered architecture.
- DBS is one of the broadest, most exciting areas in CS.

This course is important for...



- End users of DBS
- DB application programmers
- Database administrators (*DBA*)
- DBMS vendors

➡ *Must understand how a DBMS works!*

This course is prerequisite for...



- CMPT 454
Implementation of database systems
- CMPT 456
Information retrieval and web search
- CMPT 459
Data mining

Course Outline



- Introduction
- Relational Data Model
- Relational Algebra
- SQL
- Indexes
- Entity Relationship Model
- Database design
- Database applications development
- Data Warehousing and OLAP
- Data Mining

References



- Text book

H. Garcia-Molina, J. Ullman and J. Widom:
“Database Systems: The Complete Book”, Pearson
Prentice Hall, 2nd edition, 2009.

- Recommended book

R. Ramakrishnan and J. Gehrke: “Database
Management Systems”, McGraw Hill, 3rd edition,
2003.

Grading Scheme



- Assignments 40%
 - Paper and pencil assignments
 - Practical assignments with Microsoft SQL Server
- Midterm exam 20%
- Final exam 40%
- You must get an overall passing grade on the weighted average of exams in the course in order to obtain a clear pass (C- or better).